

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 #include <string>
5
6 using namespace std;
7
8 class Box {
9
10 private:
11     string name;
12     float height;
13
14 public:
15
16     Box(string n, float h) : name(n), height(h) {}
17     string getName()const {
18         return name;
19     }
20
21     float getHeight()const { return height; }
22
23     void display(int index)const {
24         cout << "#" << index + 1 << "\n"
25             << "Name: " << name << "\n"
26             << "Height: " << height << "\n\n";
27     }
28
29 };
30
31 class BoxManager {
32
33 private:
34     vector<Box> boxes;
35
36 public:
37
38     void addBox() {
39         string name;
40         float height;
41         char pass;
42
43         cin.ignore();
44         cout << "Enter Name: ";
45         getline(cin, name);
46
47
48
49
```

```
50     cout << "Enter Height: ";
51     cin >> height;
52
53
54     cin.ignore();
55
56     Box newBox(name, height);
57
58     if (boxes.empty()) {
59
60         boxes.push_back(newBox);
61     }
62
63     else
64     {
65         cout << "Add to front (F), or back (B): ";
66         cin >> pass;
67
68         if (toupper(pass) == 'F') {
69
70             boxes.insert(boxes.begin(), newBox);
71
72         }
73
74         else {
75
76             boxes.push_back(newBox);
77         }
78     }
79     cout << "Boxes Added. Total " << boxes.size() << endl << endl;
80 }
81
82 void deleteBox() {
83
84     string name;
85
86     if (boxes.empty()) {
87         cout << "The list is empty\n";
88         return;
89     }
90
91     cin.ignore();
92     cout << "Enter the name of the box to delete: ";
93     getline(cin, name);
94
95
96     for (int i = 0; i < boxes.size(); i++)
97     {
98         if (boxes[i].getName() == name)
```

```
99         {
100             swap(boxes[i], boxes.back());
101             boxes.pop_back();
102
103             cout << "The box with name " << name << " was removed.\n" <<
                "\n";
104
105             return;
106
107         }
108     }
109     cout << "\nBox with the same name " << name << " was not found." <<
        "\n";
110 }
111
112 void displayBoxes()const {
113
114     cout << "\n-----LIST INFO----\n";
115
116     cout << "The size of the list is " << boxes.size() << endl <<
        endl;
117
118     cout << "The capacity of the box is : " << boxes.capacity() <<
        endl << endl;
119
120     cout << "Max Size: " << boxes.max_size() << endl << endl;
121
122     if (boxes.empty()) {
123         cout << "No BOXES TO DISPLAY\n";
124         return;
125     }
126
127     for (int i = 0; i < boxes.size(); i++) {
128
129         boxes[i].display(i);
130
131     }
132 }
133
134 void clearAll() {
135
136     cout << "Clearing all boxes...\n";
137
138     while (!boxes.empty()) {
139         cout << boxes.back().getName() << " Was removed\n\n";
140         boxes.pop_back();
141     }
142     cout << "The list is now empty!!!!\n";
143 }
```

```
144 };
145
146
147 class Menu {
148 private:
149     BoxManager manager;
150
151
152 public:
153
154
155
156     void run() {
157         char choice{};
158
159         do {
160             cout << "\n=====MENU=====\\n";
161             cout << "A- Add Box\\n"
162                  << "B - Delete Box.\\n"
163                  << "C - Display Boxes\\n"
164                  << "D - Clear All\\n"
165                  << "Q - Quit.\\n"
166                  << "Choice? ";
167
168
169             cin >> choice;
170
171             switch (toupper(choice)) {
172                 case 'A': manager.addBox(); break;
173                 case 'B': manager.deleteBox(); break;
174                 case 'C': manager.displayBoxes(); break;
175                 case 'D': manager.clearAll(); break;
176                 case 'Q': cout << "Good Bye!!!  _ _ _ /\\n"; break;
177                 default: cout << "Invalid option/input\\n";
178             }
179
180         } while (toupper(choice) != 'Q');
181     }
182 }
183
184 };
185
186
187 int main() {
188
189     Menu app;
190
191     app.run();
192 }
```

193

194

195

196

197

198

199 return 0;

200 }